PRADITA University

IF140303-Web Application Development

Session-11:

Plugs: Authentication and Authorization

PRU/SPMI/FR-BM-18/0222

Alfa Yohannis



Contents

PRADITA University

1. Migrations
2. Models
3. Plugs
4. Signin/Signout Button
5. Routes
6. AuthController
7. Authentication in QueueController
8. Edit & Delete Buttons
9. Summary

Migration: link `queues` to `users`

PRADITA University

We generate and run a migration that adds a `user_id` foreign key to the `queues` table.

```
mix ecto.gen.migration add_user_id_to_queues
```

```
defmodule Hiwi.Repo.Migrations.AddUserIdToQueues do
  use Ecto.Migration

  def change do
    alter table(:queues) do
      add :user_id, references(:users)
    end
  end
end
```

```
mix ecto.migrate
```

Updating the `User` Schema

PRADITA University

To link users with their queues, we add a one-to-many association in the `User` schema. This enables loading all queues owned by a user without manual joins.

```
schema "users" do
  field :email, :string
  ...
  has_many :queues, Hiwi.Queue
  ...
end
```

The field `has_many` instructs Ecto that each user may own multiple queues, mapped through the `user_id` foreign key inside the `queues` table.

Updating the `Queue` Schema

PRADITA University

Each queue must reference its owner. We add a `belongs_to` relation so queues link back to the `User` schema.

```
schema "queues" do
  field :name, :string
  ...
  belongs_to :user, Hiwi.User
  ...
end
```

This association ensures data integrity and allows efficient queries when determining who owns a queue. It provides the basis for authorization logic in controllers.

SetUser Plug (Code)

PRADITA University

The `SetUser` plug loads the logged-in user from the session and assigns it to `conn.assigns`. This allows controllers and views to access `@user` without re-querying the database.

```elixir
defmodule HiwiWeb.Plugs.SetUser do
  import Plug.Conn
  alias Hiwi.Repo
  alias Hiwi.User

  def init(_params), do: nil

  def call(conn, _params) do
    user_id = get_session(conn, :user_id)

    cond do
      user = user_id && Repo.get(User, user_id) ->
        assign(conn, :user, user)
      true ->
        assign(conn, :user, nil)
    end
  end
end
```

SetUser Plug (Benefits)

PRADITA University

The plug ensures user information is consistently available for authorization and UI customization. It simplifies access to `@user` and avoids repeated database queries.

- **Efficiency**: User data is fetched once per request.
- **Easy Access**: Controllers and components use `@user`.
- **Security**: Ensures only authenticated users produce valid data.

RequireAuth Plug (Code)

PRADITA University

The `RequireAuth` plug blocks access to pages that require authentication. It checks whether `@user` exists in `conn.assigns`; if not, the request is redirected and halted.

```elixir
defmodule HiwiWeb.Plugs.RequireAuth do
  import Plug.Conn
  import Phoenix.Controller

  def init(_params), do: nil

  def call(conn, _params) do
    if conn.assigns[:user] do
      conn
    else
      conn
      |> put_flash(:error, "You haven't signed in.")
      |> redirect(to: "/")
      |> halt()
    end
  end
end
```

RequireAuth Plug (Purpose & Benefits)

The plug ensures that only authenticated users can access sensitive actions. It relies on `SetUser` to load the user.

- Ensures restricted pages are protected.
- Redirects unauthenticated users with a clear message.
- Centralizes authorization checks for multiple routes.

Layout Adjustment Based on Authentication

The layout displays different links depending on whether `@user` is present. If a user is logged in, the Signout link appears; otherwise, the GitHub login link is shown.

```
<li>
  <%= if @user do %>
    <a href={~p"/auth/signout"} class="btn btn-ghost">Signout</a>
  <% else %>
    <a href={~p"/auth/github"} class="btn btn-ghost">
      Login with GitHub
    </a>
  <% end %>
</li>
<li>
  <a href="https://phoenixframework.org/" class="btn btn-ghost">
    Website
  </a>
</li>
```

This ensures navigation reflects authentication state, improving clarity and user experience.

Authentication Routes in `router.ex`

PRADITA University

The router loads the current user through the `SetUser` plug in the `:browser` pipeline and defines authentication routes for login providers and signout handling.

```elixir
pipeline :browser do
  plug :accepts, ["html"]
  ...
  plug HiwiWeb.Plugs.SetUser
end

scope "/auth", HiwiWeb do
  pipe_through :browser
  get "/signout", AuthController, :signout
  get "/:provider", AuthController, :request
  get "/:provider/callback", AuthController, :callback
end
```

The `signout` route clears the session safely, while provider routes handle external authentication flows.

Handling Signout in `AuthController`

The `signout` function clears the session and redirects the user to the homepage, ensuring the authentication state is fully reset.

```elixir
def signout(conn, _params) do
  conn
  |> configure_session(drop: true)
  |> put_flash(:info, "Signout has been successful.")
  |> redirect(to: "/")
end
```

Clearing the session prevents unauthorized reuse, while the flash message confirms that signout was successful.

Authorization in `QueueController`

PRADITA University

The controller restricts sensitive actions using `RequireAuth` and validates ownership with `:check_queue_owner`. Only logged-in users—and only the queue owner—may modify or delete queues.

```
...
plug HiwiWeb.Plugs.RequireAuth
  when action in [:new, :create, :update, :edit, :delete]

plug :check_queue_owner
  when action in [:update, :edit, :delete]
...
def create(conn, %{"queue" => queue_params}) do
  changeset =
    conn.assigns.user
    |> Ecto.build_assoc(:queues)
    |> Queue.changeset(queue_params)
  ...
end
```

Create Action with Authorization

PRADITA University

The `create` action attaches the new queue to the logged-in user via `Ecto.build_assoc`. Successful inserts redirect with a flash message; errors re-render the form. This ensures new queues always have a valid owner and integrates cleanly with later authorization checks.

Benefits:

- Strong protection for sensitive actions.
- Ensures only owners modify their queues.
- Clear feedback through consistent flash messages.

Showing Edit/Delete Links Only to Owners

The view displays `Edit` and `Delete` links only when the logged-in user owns the queue. This is checked by comparing `@user.id` with `q.user_id`.

```
<div class="join">
  <.link navigate={~p"/queues/#{q.id}"}
         class="btn btn-ghost btn-xs join-item">View</.link>

  <%= if @user && @user.id == q.user_id do %>
    <.link navigate={~p"/queues/edit/#{q.id}"}
           class="btn btn-ghost btn-xs join-item">Edit</.link>
    <.link href={~p"/queues/delete/#{q.id}"} method="delete"
           data-confirm="Delete this queue?"
           class="btn btn-ghost btn-xs text-error join-item">
      Delete
    </.link>
  <% end %>
</div>
```

This prevents unauthorized users from seeing or triggering sensitive actions.

Summary: Authentication & Authorization

- Phoenix plugs process requests and enable authentication/authorization.
- Ecto associations (`User–Queue`) establish data ownership.
- `SetUser` loads the logged-in user into `conn.assigns`.
- `RequireAuth` restricts actions to authenticated users.
- Controllers verify queue ownership before sensitive actions.
- Views conditionally display edit/delete actions only for owners.
- Combined logic ensures data integrity and enforces secure access control.